DEVOPS PRACTICAL

ASSIGNMENTS

Student Assignment Brief

Linux • Bash • Networking • Git • Docker • GitHub Actions

Three practical assignments to be completed locally and submitted through GitHub.
No cloud deployment is required.

Each assignment includes a local grade.sh script that will be used as part of the grading process.

1. Submission Requirements

• Create a separate GitHub repository for each assignment.

• Submit the repository URL to the instructor.

• The instructor must be able to clone the repository and run the provided grading script locally.

• Include a README.md with installation/setup, usage, testing and any assumptions. • Do not commit passwords, tokens, private keys or other secrets.

• Scripts must work in a Linux environment and should not depend on hardcoded machine-specific values.

• Use meaningful Git commits and keep the repository organized.

SUBMISSION LINK:

https://docs.google.com/forms/d/e/1FAIpQLSfNRfSI_-x6mo6czu0J44a2thkdqGTZGzgtXOA8KVYGklQ7-A/viewform?usp=header

Before submitting

```
git clone <your-repository-url>
cd <repository>
chmod +x *.sh app/*.sh scripts/*.sh tests/*.sh 2>/dev/null || true
./grade.sh
```

The instructor may run additional manual tests after grade.sh completes.

2. Assignment 1 — Linux, Bash & Networking

Build a Linux diagnostic toolkit using Bash. The scripts should collect system information, check disk usage and perform basic network checks.

Required structure

```
assignment-1/
■■■ README.md
■■■ system-info.sh
■■■ disk-check.sh
■■■ network-check.sh
■■■ grade.sh
■■■ logs/
 ■■■ .gitkeep
```

system-info.sh

Display hostname, current user, date/time, operating system, kernel version, uptime, CPU information, memory information and current working directory. Values must be obtained from the system at runtime.

disk-check.sh

```
./disk-check.sh <threshold> [path]
```

• Default path: /.

• Threshold must be an integer from 1 to 100.

• Display the disk usage percentage.

• Exit 0 when usage is below the threshold.

• Exit non-zero when usage reaches or exceeds the threshold.

• Exit 2 for invalid input.

network-check.sh

```
./network-check.sh <hostname-or-ip> [port]
```

• Validate the host argument.

• Resolve the host and display the resolved address.

• Perform a basic connectivity check.

• Display network interface information.

• If a port is supplied, check TCP connectivity.

• Valid ports are 1–65535.

• Invalid input must return a non-zero result and should not crash the script.

Logging

Create useful logs under logs/. Entries should contain a timestamp and a description of the operation.

Git

• At least 5 meaningful commits.

• Use at least one non-main feature branch.

• Merge the feature branch into main/master.

• Use clear commit messages.

grade.sh

The supplied Assignment 1 grader checks required files, Bash syntax, executable permissions, system-info output, disk argument validation, network validation, logging and basic Git history.

```
chmod +x grade.sh *.sh
./grade.sh
```

Grading

Area	Points
Linux commands	20

Bash scripting	25
Networking	20
Error handling	10
Logging	10
Git workflow	10
README	5
Total	**100**

3. Assignment 2 — Dockerized Diagnostic CLI

Turn a Bash diagnostic tool into a Dockerized application that can be built and run locally.

Required structure

```
assignment-2/
■■■ README.md
■■■ app/
■ ■■■ diagnostic.sh
■ ■■■ health-check.sh
■■■ Dockerfile
■■■ compose.yaml
■■■ .dockerignore
■■■ test.sh
■■■ grade.sh
```

CLI commands

```
diagnostic system
diagnostic network <host>
diagnostic disk
diagnostic help
```

• system: display useful Linux system information.

• network: check the supplied host.

• disk: display disk information.

• help: display commands and usage.

• Invalid commands and missing arguments must be handled properly.

Exit codes

• 0 — success

• 1 — operational/runtime failure

• 2 — invalid command or input

Dockerfile

• Use a lightweight Linux base image.

• Install only required packages.

• Copy the application into the image.

• Set executable permissions.

• Configure ENTRYPOINT and/or CMD.

```
docker build -t diagnostic-tool .
docker run --rm diagnostic-tool system
docker run --rm diagnostic-tool disk
docker run --rm diagnostic-tool help
```

.dockerignore

Exclude unnecessary files such as .git, logs and temporary/local

files. **Docker Compose**

Provide a compose.yaml that allows the application to be run locally with Docker

Compose. `docker compose run --rm diagnostic system`

test.sh

Create tests for the Docker image covering at least help, system, disk and invalid command handling.

grade.sh

The supplied Assignment 2 grader checks the project structure, Bash syntax, executable permissions, Dockerfile, .dockerignore, image build, basic container commands, invalid command handling, Docker Compose configuration and the student test suite.

```
chmod +x grade.sh test.sh app/*.sh
./grade.sh
```

Grading

Area	Points
Bash CLI	20
Linux functionality	10
Error handling	10
Dockerfile	20
Docker practices	10
Docker Compose	10
Testing	10
Git workflow	5
README	5
Total	**100**

4. Assignment 3 — CI/CD with GitHub Actions

Build a local CI pipeline for a Bash application. The pipeline must validate the code, run tests and build/smoke-test a Docker image. There is no cloud deployment.

Required structure

```
assignment-3/
■■■ README.md
■■■ app/
■ ■■■ app.sh
■■■ scripts/
■ ■■■ lint.sh
■ ■■■ build.sh
■■■ tests/
■ ■■■ test.sh
■■■ .github/
■ ■■■ workflows/
■ ■■■ ci.yml
■■■ Dockerfile
■■■ compose.yaml
■■■ .dockerignore
■■■ grade.sh
```

Application

```
./app/app.sh system-info
./app/app.sh check-host <host>
./app/app.sh check-port <host> <port>
./app/app.sh help
```

• system-info: display system information.

• check-host: resolve/check a host.

• check-port: validate the port and check TCP connectivity.

• help: display usage.

• Invalid input should return exit code 2.

lint.sh

Check that required files exist and run bash -n against the Bash scripts. ShellCheck can be added as an extra check.

tests/test.sh

Write at least 8 meaningful tests covering help, system-info, invalid commands, missing host, a valid host, missing port, non-numeric port and out-of-range ports.

Docker

```
docker build -t devops-tool .
docker run --rm devops-tool help
docker run --rm devops-tool system-info
```

scripts/build.sh should build the image and perform Docker smoke tests, including an invalid-command test.

GitHub Actions

Create .github/workflows/ci.yml. It must run on push and pull_request.

```
validate
|
v
test
|
v
docker
```

• validate runs linting.
• test runs after validate succeeds.

• docker runs after test succeeds.

• Use needs: to enforce the order.

CI failure demonstration

Create a branch, introduce a syntax error or failing test, push it and show that CI fails. Fix the issue and push again. The final workflow must pass. Document this briefly in README.md.

grade.sh

The supplied Assignment 3 grader checks the repository structure, Bash syntax, executable permissions, workflow triggers, required jobs, job dependencies, application behaviour, linting, Docker build/smoke tests, student tests and basic Git history.

```
chmod +x grade.sh app/*.sh scripts/*.sh tests/*.sh
./grade.sh
```

Grading

Area	Points
Bash scripting	10
Linux/networking	10
Automated tests	15
Dockerfile	15
Docker practices	10
GitHub Actions	20
Job dependencies	5
Error handling	5
Git workflow	5
README	5
Total	**100**

5. What the Instructor Will Do

• Clone the submitted repository.

• Run the assignment's grade.sh.

• Review the test output.

• Run additional manual tests where necessary.

• Inspect the code and repository structure.

• Inspect Git history.

• For Assignment 3, inspect the GitHub Actions workflow and CI history.

• Apply the relevant 100-point rubric.

Important

Passing grade.sh does not automatically mean a score of 100. The script handles repeatable checks; code quality, Git practices, documentation, Docker configuration and other requirements may be reviewed manually.